

Application Layer Activity & Protocol Visualizer

Comprehensive Technical Documentation, Protocol RFC Models, Local Setup & Cloud Deployment Guide

Project Repository:	https://github.com/anuragkash1646-dotcom/Application-dashboard
Live Cloud Deployment:	https://application-dashboard-production.up.railway.app
Healthcheck Endpoint:	https://application-dashboard-production.up.railway.app/health
Technology Stack:	Python 3.10+, FastAPI, Uvicorn, WebSockets, Pydantic, Vanilla ES6 JavaScript, HTML5/CSS3

1. Project Overview & Core Goals

The **Application Layer Activity & Protocol Visualizer** is an interactive educational Computer Networks Layer-7 protocol simulation suite. It bridges the conceptual gap between everyday user actions (such as browsing a website, sending an email, or streaming a video) and the exact RFC-defined protocol dialogues that execute beneath the surface.

Unlike abstract flowcharts or mock interfaces, this system operates a **deterministic RFC packet engine** coupled with a **real-time WebSocket synchronization layer**. Any state change—whether triggering a web visit, pausing the transmission, stepping forward, or jumping across packet timelines—is broadcast in real time to all connected browser clients.

2. System Architecture & File Structure

The application is structured into decoupled components following clean software engineering principles:

FastAPI Server	app/main.py	Mounts static dashboard SPA, hosts /health healthcheck, and handles WebSocket handshakes at /ws.
WebSocket Manager	app/websocket_manager.py	Manages active client pool, broadcast messaging, shared simulation state, and an asynchronous timer loop for automated playback.
Protocol Simulator	app/protocol_simulator.py	Deterministic packet generation for DNS (RFC 1035), HTTP/1.1 (RFC 2616/7230), SMTP (RFC 5321), and HLS (RFC 8216).
Web Dashboard	app/static/index.html	Dual-panel interface: Left panel (Activity forms, logs); Right panel (Client ↔ Server nodes, wire inspector); Bottom timeline.
Dashboard Styling	app/static/style.css	Modern dark cybersecurity aesthetic, flying packet keyframe animations (C2S/S2C), glowing nodes, and responsive layout.
Frontend Controller	app/static/app.js	Vanilla ES6 WebSocket client: handles auto-reconnect, packet visualization, tab switching, and keyboard shortcuts.
Test Suite	tests/test_protocol_simulator.py	7 automated pytest cases covering health, DNS, HTTP, SMTP state machine, HLS manifest/segments, and WebSockets.
Deployment Config	Procfile & railway.json	Zero-config Nixpacks build declaration, startup command, and automated healthcheck for Railway cloud deployment.

3. Detailed Protocol Simulation Specifications

3.1 Web Browsing Simulation (DNS + TCP + HTTP/1.1)

Entering a URL (e.g. `https://example.com/index.html`) triggers a multi-tier resolution and fetch sequence:

- **DNS Query (UDP Port 53):** Standard query with Transaction ID, A-record lookup, and Recursion Desired (RD) flag set to 0x0100.
- **DNS Response (UDP Port 53):** Server returns resolved IP address (e.g. 93.184.216.34) and TTL 300 seconds with NOERROR.
- **TCP 3-Way Handshake (Port 80):** Client initiates SYN, server responds with SYN, ACK, and client confirms with ACK.
- **HTTP/1.1 GET Request:** Transmits GET /path HTTP/1.1 with headers: Host, User-Agent, Accept, and Connection: keep-alive.
- **HTTP 200 OK Response:** Delivers server timestamp, Content-Type: text/html, Content-Length, and rendered HTML payload.

3.2 SMTP Email Dispatch Simulation (RFC 5321)

Simulates an email dispatch over TCP Port 25 following the full SMTP state machine:

1. **220 Service Ready:** Server welcomes client (Postfix ESMTP ready).
2. **EHLO client.fqdn:** Client identifies its fully qualified domain name.
3. **250 ESMTP:** Server advertises capabilities (PIPELINING, SIZE 10485760, 8BITMIME).
4. **MAIL FROM:<sender>:** Specifies envelope sender address.
5. **250 2.1.0 Ok:** Server accepts sender address.
6. **RCPT TO:<recipient>:** Specifies mailbox recipient address.
7. **250 2.1.5 Ok:** Server accepts recipient.
8. **DATA:** Client requests permission to transmit body.
9. **354 Start Input:** Server responds to begin input, ending with <CRLF>.<CRLF>.
10. **MIME Payload:** Transmits RFC headers (From, To, Date, Subject, Message-ID) and body.
11. **250 2.0.0 Ok queued:** Server assigns queue ID for dispatch.
12. **QUIT:** Client initiates session teardown.
13. **221 Bye:** Server closes transmission channel.

3.3 HLS Adaptive Video Streaming Simulation (RFC 8216)

Simulates chunk-based HTTP Live Streaming with dynamic client buffer telemetry:

- **CDN Hostname Resolution:** UDP 53 DNS resolution for CDN edge host (cdn.videoedge.net).
- **Master Playlist Request (master.m3u8):** Delivers available bitrate variants (1080p @ 5 Mbps, 720p @ 2.5 Mbps, 360p @ 800 Kbps).
- **Variant Playlist Request (720p/playlist.m3u8):** Delivers target duration (4 seconds) and segment list.
- **Progressive 4-Second MPEG-TS Chunk Requests:** Progressively requests segment_000.ts through segment_003.ts.
- **Dynamic Buffer Health Telemetry:** Client buffer increases with each segment received (0.0s → 4.0s → 8.0s → 12.0s → 16.0s), visualised on the dashboard buffer meter.

4. Step-by-Step Instructions to Run Locally

Step 4.1: Prerequisites

Ensure **Python 3.10 or higher** and **Git** are installed on your system.

Step 4.2: Clone the Repository

```
git clone https://github.com/anuragkash1646-dotcom/Application-dashboard.git
cd Application-dashboard
```

Step 4.3: Install Dependencies

Install the required Python libraries using pip:

```
pip install -r requirements.txt
```

Step 4.4: Launch the Visualizer Server

Start the FastAPI application using Uvicorn:

```
uvicorn app.main:app --reload --host 127.0.0.1 --port 8000
```

Step 4.5: Open Dashboard & Verify Endpoints

- **Interactive Dashboard:** Open <http://127.0.0.1:8000/> in your web browser.
- **API Healthcheck:** Open <http://127.0.0.1:8000/health> (returns {"status":"ok"}).

5. Dashboard Controls & Keyboard Navigation

Play / Pause Toggle	Spacebar	Freezes or resumes automatic packet playback ticker.
Step Next	Right Arrow (➡)	Advances simulation exactly one packet forward and updates wire inspector.
Step Previous	Left Arrow (⬅)	Steps back exactly one packet in the historical transaction.
Replay	R key	Resets simulation to packet 1 and restarts automatic playback.
Timeline Jump	Click circular node	Directly jumps to any numbered event node (1..N).
Playback Speed	Dropdown (0.5x – 2.0x)	Dynamically adjusts delay interval between automatic packet steps.

6. Automated Testing Suite

The project includes a comprehensive pytest suite verifying all protocol generation logic, state transitions, and WebSocket lifecycles.

Execute the test suite with verbose output:

```
pytest tests/ -v
```

test_health_endpoint	Verifies GET /health returns HTTP 200 with {"status":"ok"}	PASSED
test_index_dashboard_serves_html	Verifies root URL / successfully serves HTML dashboard	PASSED
test_dns_event_generation	Validates UDP 53, A-record query, RD flag 0x0100, resolved IP, and TTL 300	PASSED
test_http_transaction	Validates HTTP/1.1 GET request headers, Port 80, 200 OK, Content-Type, and HTML body	PASSED
test_smtp_state_machine	Validates complete RFC 5321 conversation (220 ➔ EHLO ➔ DATA ➔ QUIT ➔ 221)	PASSED
test_hls_manifest_and_segments	Validates master/variant manifests and progressive buffer health (4s ➔ 8s ➔ 12s ➔ 16s)	PASSED
test_websocket_lifecycle_and_actions	Validates WebSocket connection, initial snapshot, pause, next, prev, jump, and replay	PASSED

7. Cloud Deployment Guide (Railway)

The repository is configured for zero-configuration, continuous deployment on Railway using Nixpacks.

1. **Login:** Sign in to <https://railway.app> using your GitHub account.
2. **New Project:** Click '+ New Project' ➔ 'Deploy from GitHub repo'.
3. **Select Repository:** Choose **anuragkash1646-dotcom/Application-dashboard**.
4. **Automatic Build:** Railway reads railway.json and installs packages from requirements.txt.
5. **Public Domain:** Under Service Settings ➔ Networking, generate a public domain to obtain your live deployment URL.

8. Direct Access Links Reference

Resource	URL (Clickable Link)	Notes
Live Application	https://application-dashboard-production.up.railway.app	Production deployment running on Railway
GitHub Repository	https://github.com/anuragkash1646-dotcom/Application-dashb-oard	Source code, commits, and issue tracking
Live Healthcheck	https://application-dashboard-production.up.railway.app/health	Healthcheck returning status: ok